

PYTHON CODE:

```
# Bisection Method
import math
equation = input("Enter the equation in x: ")
def f(x):
    return eval(equation)
x1 = float(input("Enter first guess: "))
x2 = float(input("Enter second guess: "))
tolerance = float(input("Enter tolerance value: "))
iterations = 0
def end():
    print("Root =", ('%.6f' % root).rstrip('0').rstrip('.'))
    print("Number of iterations =", ('%.6f' %
iterations).rstrip('0').rstrip('.'))
    exit()
if f(x1) == 0:
    root = x1
    end()
elif f(x2) == 0:
    root = x2
    end()
elif f(x1) * f(x2) > 0:
    print("Invalid Interval")
else:
    while abs(x1 - x2) > tolerance:
        x0 = (x1 + x2) / 2
        iterations += 1
        if f(x0) == 0:
            break
        elif f(x1) * f(x0) < 0:
            x2 = x0
        else:
            x1 = x0
    root = x0
    end()
```

INPUT:

```
Enter the equation in x: x**3-4*x-9
Enter first guess: 2
Enter second guess: 3
Enter tolerance value: 0.000001
```

OUTPUT:

```
Root = 2.706529
Number of iterations = 20
```

PYTHON CODE:

Regula-Falsi Method

```
import math
equation = input("Enter the equation in x: ")
def f(x):
    return eval(equation)
x1 = float(input("Enter first guess: "))
x2 = float(input("Enter second guess: "))
tolerance = float(input("Enter tolerance value: "))
iterations = 0
def end():
    print("Root =", ('%.6f' % root).rstrip('0').rstrip('.'))
    print("Number of iterations =", ('%.6f' %
iterations).rstrip('0').rstrip('.'))
    exit()
if f(x1) == 0:
    root = x1
    end()
elif f(x2) == 0:
    root = x2
    end()
elif f(x1) * f(x2) >= 0:
    print("Invalid Interval")
else:
    while abs(x1 - x2) > tolerance:
        x0 = ((x1 * f(x2)) - (x2 * f(x1))) / (f(x2) - f(x1))
        iterations += 1
        if f(x0) == 0:
            break
        elif f(x1) * f(x0) < 0:
            x2 = x0
        else:
            x1 = x0
    root = x0
    end()
```

INPUT:

```
Enter the equation in x: x**3-4*x-9
Enter first guess: 2
Enter second guess: 3
Enter tolerance value: 0.000001
```

OUTPUT:

```
Root = 2.706528
Number of iterations = 18
```

PYTHON CODE:

Newton-Raphson Method

```
import math
equation = input("Enter the equation in x: ")
derivative = input("Enter derivative of equation: ")
def f(x):
    return eval(equation)
def df(x):
    return eval(derivative)
x0 = float(input("Enter initial guess: "))
tolerance = float(input("Enter tolerance value: "))
iterations = 0
maxiterations = 200
def end():
    print("Root =", ('%.6f' % x1).rstrip('0').rstrip('.'))
    print("Number of iterations =", ('%.6f' %
iterations).rstrip('0').rstrip('.'))
    exit()
if f(x0) == 0:
    x1 = x0
    end()
elif df(x0) == 0:
    print("Invalid Interval")
    exit()
while abs(f(x0)) > tolerance:
    x1 = x0 - f(x0) / df(x0)
    x0 = x1
    iterations += 1
    if iterations == maxiterations:
        print("Maximum iterations reached. Method failed to converge.")
        exit()
end()
```

INPUT:

```
Enter the equation in x: x**3-4*x-9
Enter derivative of equation: 3*x**2-4
Enter initial guess: 2
Enter tolerance value: 0.000001
```

OUTPUT:

```
Root = 2.706528
Number of iterations = 5
```

PYTHON CODE:

```
# Secant Method
import math
equation = input("Enter the equation in x: ")
def f(x):
    return eval(equation)
x1 = float(input("Enter first guess: "))
x2 = float(input("Enter second guess: "))
tolerance = float(input("Enter tolerance value: "))
iterations = 0
maxiterations = 200
def end():
    print("Root =", ('%.6f' % x3).rstrip('0').rstrip('.'))
    print("Number of iterations =", ('%.6f' %
iterations).rstrip('0').rstrip('.'))
    exit()
if f(x1) == 0:
    x3 = x1
    end()
elif f(x2) == 0:
    x3 = x2
    end()
if f(x2) - f(x1) == 0:
    print("Method failed to converge. Choose different points.")
while abs(x1 - x2) > tolerance:
    x3 = ((x1 * f(x2)) - (x2 * f(x1))) / (f(x2) - f(x1))
    x1 = x2
    x2 = x3
    iterations += 1
    if iterations == maxiterations:
        print("Method failed to converge. Maximum iterations reached.")
        exit()
end()
```

INPUT:

```
Enter the equation in x: x**3-4*x-9
Enter first guess: 2
Enter second guess: 3
Enter tolerance value: 0.000001
```

OUTPUT:

```
Root = 2.706528
Number of iterations = 6
```

PYTHON CODE:

```
# Gauss Elimination Method
n = int(input("Enter number of variables: "))
a = []
print("Enter augmented matrix coefficients:")
for i in range(n):
    row = list(map(float, input().split()))
    if len(row) != n + 1:
        print(f"Error: Row {i+1} must contain {n+1} values (got {len(row)}).")
        exit()
    a.append(row)

# Forward Elimination
for i in range(n):
    for j in range(i + 1, n):
        ratio = a[j][i] / a[i][i]
        for k in range(n + 1):
            a[j][k] = a[j][k] - ratio * a[i][k]

# Back Substitution
x = [0 for i in range(n)]
x[n - 1] = a[n - 1][n] / a[n - 1][n - 1]
for i in range(n - 2, -1, -1):
    sum = 0
    for j in range(i + 1, n):
        sum = sum + a[i][j] * x[j]
    x[i] = (a[i][n] - sum) / a[i][i]
print("\nSolution:")
for i in range(n):
    val = ('%.6f' % x[i]).rstrip('0').rstrip('.')
    print(f"x{i+1} = {val}")
```

INPUT:

```
Enter number of variables: 4
Enter augmented matrix coefficients:
10 -7 3 5 6
-6 8 -1 -4 5
3 1 4 11 2
5 -9 -2 4 7
```

OUTPUT:

```
Solution:
x1 = 5
x2 = 4
x3 = -7
x4 = 1
```

PYTHON CODE:

```
# Gauss Jordan Method
n = int(input("Enter number of variables: "))
a = []
print("Enter augmented matrix coefficients:")
for i in range(n):
    row = list(map(float, input().split()))
    a.append(row)
# Gauss Jordan Elimination
for i in range(n):
    pivot = a[i][i]
    for j in range(n + 1):
        a[i][j] = a[i][j] / pivot
    for k in range(n):
        if k != i:
            factor = a[k][i]
            for j in range(n + 1):
                a[k][j] = a[k][j] - factor * a[i][j]
print("Solution:")
for i in range(n):
    val = ('%.6f' % a[i][n]).rstrip('0').rstrip('.')
    print(f"x{i+1} = {val}")
```

INPUT:

```
Enter number of variables: 4
Enter augmented matrix coefficients:
1 2 3 -1 10
2 3 -3 -1 1
2 -1 2 3 7
3 2 -4 3 2
```

OUTPUT:

```
Solution:
x1 = 1
x2 = 2
x3 = 2
x4 = 1
```

PYTHON CODE:

```
# Gauss-Jacobi Method
n = int(input("Enter number of variables: "))
a = []
print("Enter augmented matrix coefficients:")
for i in range(n):
    row = list(map(float, input().split()))
    a.append(row)
coeff = [row[:n] for row in a]
b = [row[n] for row in a]
print("Enter initial guesses:")
x = list(map(float, input().split()))
tolerance = float(input("Enter tolerance value: "))
def is_diagonally_dominant(matrix, n):
    for i in range(n):
        diag = abs(matrix[i][i])
        off_diag_sum = sum(abs(matrix[i][j]) for j in range(n) if j != i)
        if diag < off_diag_sum:
            return False
    return True
if is_diagonally_dominant(coeff, n):
    print("\nMatrix is diagonally dominant")
else:
    print("\nWarning: Matrix is NOT diagonally dominant")
    print("Gauss-Jacobi method will not converge for this system.")
    exit()
x_new = [0 for _ in range(n)]
iteration_count = 0
while True:
    iteration_count += 1
    for i in range(n):
        total = 0
        for j in range(n):
            if i != j:
                total += coeff[i][j] * x[j]
        x_new[i] = (b[i] - total) / coeff[i][i]
    error = max(abs(x_new[i] - x[i]) for i in range(n))
    x = x_new.copy()
    if error < tolerance:
        break
print(f"\nConverged in {iteration_count} iterations.")
print("\nSolution:")
for i in range(n):
    val = x[i]
    if abs(val - round(val)) < tolerance:
        val_str = str(int(round(val)))
```

```
else:
    val_str = ('%.6f' % val).rstrip('0').rstrip('.')
    print(f"x{i+1} = {val_str}")
```

INPUT:

```
Enter number of variables: 4
Enter augmented matrix coefficients:
10 -2 -1 -1 3
-2 10 -1 -1 15
-1 -1 10 -2 27
-1 -1 -2 10 -9
Enter initial guesses:
0 0 0 0
Enter tolerance value: 0.000001
```

OUTPUT:

```
Matrix is diagonally dominant

Converged in 16 iterations.

Solution:
x1 = 1
x2 = 2
x3 = 3
x4 = 0
```


PYTHON CODE:

```
# Gauss-Seidel Method
n = int(input("Enter number of variables: "))
a = []
print("Enter augmented matrix coefficients:")
for i in range(n):
    row = list(map(float, input().split()))
    a.append(row)
coeff = [row[:n] for row in a]
b = [row[n] for row in a]
print("Enter initial guesses:")
x = list(map(float, input().split()))
tolerance = float(input("Enter tolerance value: "))
def is_diagonally_dominant(matrix, n):
    for i in range(n):
        diag = abs(matrix[i][i])
        off_diag_sum = sum(abs(matrix[i][j]) for j in range(n) if j != i)
        if diag < off_diag_sum:
            return False
    return True
if is_diagonally_dominant(coeff, n):
    print("\nMatrix is diagonally dominant")
else:
    print("\nWarning: Matrix is NOT diagonally dominant")
    print("Gauss-Seidel method will not converge for this system.")
    exit()
iteration_count = 0
while True:
    iteration_count += 1
    max_error = 0
    for i in range(n):
        old = x[i]
        total = 0
        for j in range(n):
            if i != j:
                total += coeff[i][j] * x[j]
        x[i] = (b[i] - total) / coeff[i][i]
        if abs(x[i] - old) > max_error:
            max_error = abs(x[i] - old)
    if max_error < tolerance:
        break
print(f"\nConverged in {iteration_count} iterations.")
print("\nSolution:")
for i in range(n):
    val = x[i]
    if abs(val - round(val)) < tolerance:
```

```
        val_str = str(int(round(val)))
    else:
        val_str = ('%.6f' % val).rstrip('0').rstrip('.')
    print(f"x{i + 1} = {val_str}")
```

INPUT:

```
Enter number of variables: 4
Enter augmented matrix coefficients:
10 -2 -1 -1 3
-2 10 -1 -1 15
-1 -1 10 -2 27
-1 -1 -2 10 -9
Enter initial guesses:
0 0 0 0
Enter tolerance value: 0.000001
```

OUTPUT:

```
Matrix is diagonally dominant

Converged in 10 iterations.

Solution:
x1 = 1
x2 = 2
x3 = 3
x4 = 0
```

PYTHON CODE:

```
# Lagrange's Interpolation
n = int(input("Enter number of data points: "))
x = []
y = []
print("Enter x and y values:")
for i in range(n):
    xi, yi = map(float, input().split())
    x.append(xi)
    y.append(yi)
choice = int(input("Enter 1 for y corresponding to x\nEnter 2 for x\n\n"))
# Direct Interpolation
if choice == 1:
    value = float(input("Enter value of x: "))
    result = 0
    for i in range(n):
        term = y[i]
        for j in range(n):
            if i != j:
                term = term * (value - x[j]) / (x[i] - x[j])
        result = result + term
    print("Interpolated value of y =", ('%.6f' %
result).rstrip('0').rstrip('.'))
# Inverse Interpolation
elif choice == 2:
    value = float(input("Enter value of y: "))
    result = 0
    for i in range(n):
        term = x[i]
        for j in range(n):
            if i != j:
                term = term * (value - y[j]) / (y[i] - y[j])
        result = result + term
    print("Interpolated value of x =", ('%.6f' %
result).rstrip('0').rstrip('.'))
else:
    print("Invalid Choice")
```

DIRECT INTERPOLATION:

INPUT:

```
Enter number of data points: 5
Enter x and y values:
5 150
7 392
11 1452
13 2366
17 5202
Enter 1 for y corresponding to x
Enter 2 for x corresponding to y
1
Enter value of x: 9
```

OUTPUT:

```
Interpolated value of y = 810
```

INVERSE INTERPOLATION:

INPUT:

```
Enter number of data points: 5
Enter x and y values:
5 150
7 392
11 1452
13 2366
17 5202
Enter 1 for y corresponding to x
Enter 2 for x corresponding to y
2
Enter value of y: 810
```

OUTPUT:

```
Interpolated value of x = 9.2331057152
```

PYTHON CODE:

```
# Newton's Forward/Backward Interpolation
n = int(input("Enter number of data points: "))
x = []
y = []
print("Enter x and y values:")
for i in range(n):
    xi, yi = map(float, input().split())
    x.append(xi)
    y.append(yi)
value = float(input("\nEnter value of x: "))
equal_spacing = True
h = x[1] - x[0]
for i in range(1, n - 1):
    if abs((x[i+1] - x[i]) - h) > 1e-9:
        equal_spacing = False
        break
if not equal_spacing:
    print("\nError: Intervals are not equally spaced. Newton's
interpolation cannot be applied.")
    exit()
diff = [[0 for j in range(n)] for i in range(n)]
for i in range(n):
    diff[i][0] = y[i]
for j in range(1, n):
    for i in range(n - j):
        diff[i][j] = diff[i + 1][j - 1] - diff[i][j - 1]
def format_number(num, width=12):
    s = ('%.6f' % num).rstrip('0').rstrip('.')
    return f"{s:>{width}}"
print("\nDifference Table:")
headers = ["x", "y"] + [f" $\Delta^j y$ " for j in range(1, n)]
for hname in headers:
    print(f"{hname:>12}", end="")
print()
for i in range(n):
    print(format_number(x[i]), end="")
    for j in range(n - i):
        print(format_number(diff[i][j]), end="")
    print()
if abs(value - x[0]) <= abs(value - x[-1]):
    # Forward interpolation
    print("\nUsing Forward Interpolation")
    p = (value - x[0]) / h
    result = y[0]
    p_term = 1
```

```

factorial = 1
for i in range(1, n):
    p_term *= (p - i + 1)
    factorial *= i
    result += (p_term * diff[0][i]) / factorial
else:
    # Backward interpolation
    print("\nUsing Backward Interpolation")
    p = (value - x[-1]) / h
    result = y[-1]
    p_term = 1
    factorial = 1
    for i in range(1, n):
        p_term *= (p + i - 1)
        factorial *= i
        result += (p_term * diff[n - i - 1][i]) / factorial
print("Interpolated value =", ('%.6f' % result).rstrip('0').rstrip('.'))

```

INPUT:

FORWARD INTERPOLATION:

```

Enter number of data points: 7
Enter x and y values:
100 10.63
150 13.03
200 15.04
250 16.81
300 18.42
350 19.90
400 21.27

Enter value of x: 160

```

BACKWARD INTERPOLATION:

```

Enter number of data points: 7
Enter x and y values:
100 10.63
150 13.03
200 15.04
250 16.81
300 18.42
350 19.90
400 21.27

Enter value of x: 410

```

OUTPUT:

DIFFERENCE TABLE:

Difference Table:							
x	y	Δ^1y	Δ^2y	Δ^3y	Δ^4y	Δ^5y	Δ^6y
100	10.63	2.4	-0.39	0.15	-0.07	0.02	0.02
150	13.03	2.01	-0.24	0.08	-0.05	0.04	
200	15.04	1.77	-0.16	0.03	-0.01		
250	16.81	1.61	-0.13	0.02			
300	18.42	1.48	-0.11				
350	19.9	1.37					
400	21.27						

Using Forward Interpolation
Interpolated value = 13.457333

Using Backward Interpolation
Interpolated value = 21.535246

PYTHON CODE:

```
# Trapezoidal Rule
n = int(input("Enter number of intervals: "))
x = []
y = []
print("Enter x and y values:")
for i in range(n + 1):
    xi, yi = map(float, input().split())
    x.append(xi)
    y.append(yi)
equal_spacing = True
h = x[1] - x[0]
for i in range(1, n):
    if abs((x[i+1] - x[i]) - h) > 1e-9: # tolerance for floating-point
        equal_spacing = False
        break
if not equal_spacing:
    print("\nError: Intervals are not equally spaced. Trapezoidal rule
cannot be applied.")
else:
    sum = y[0] + y[n]
    for i in range(1, n):
        sum = sum + 2 * y[i]
    result = (h / 2) * sum
print("Value of integration =", ('%.6f' %
result).rstrip('0').rstrip('.'))
```

INPUT:

```
Enter number of intervals: 6
Enter x and y values:
0 1
1 0.5
2 0.2
3 0.1
4 0.0588
5 0.0385
6 0.027
```

OUTPUT:

```
Value of integration = 1.4108
```

PYTHON CODE:

```
# Simpson's 1/3rd Rule
n = int(input("Enter number of intervals (even): "))
if n % 2 != 0:
    print("Error: Simpson's 1/3 rule requires an even number of intervals.")
    exit()
x = []
y = []
print("Enter x and y values:")
for i in range(n + 1):
    xi, yi = map(float, input().split())
    x.append(xi)
    y.append(yi)
equal_spacing = True
h = x[1] - x[0]
for i in range(1, n):
    if abs((x[i+1] - x[i]) - h) > 1e-9: # tolerance for floating-point
        equal_spacing = False
        break
if not equal_spacing:
    print("\nError: Intervals are not equally spaced. Trapezoidal rule cannot be applied.")
else:
    sum = y[0] + y[n]
    for i in range(1, n):
        if i % 2 == 0:
            sum = sum + 2 * y[i]
        else:
            sum = sum + 4 * y[i]
    result = (h / 3) * sum
    print("Value of integration =", ('%.6f' % result).rstrip('0').rstrip('.'))
```

INPUT:

```
Enter number of intervals (even): 6
Enter x and y values:
0 1
1 0.5
2 0.2
3 0.1
4 0.0588
5 0.0385
6 0.027
```

OUTPUT:

```
Value of integration = 1.3662
```


PYTHON CODE:

```
# Simpson's 3/8th Rule
n = int(input("Enter number of intervals (multiple of 3): "))
if n % 3 != 0:
    print("Error: Simpson's 3/8 rule requires number of intervals to be a
multiple of 3.")
    exit()
x = []
y = []
print("Enter x and y values:")
for i in range(n + 1):
    xi, yi = map(float, input().split())
    x.append(xi)
    y.append(yi)
equal_spacing = True
h = x[1] - x[0]
for i in range(1, n):
    if abs((x[i+1] - x[i]) - h) > 1e-9: # tolerance for floating-point
        equal_spacing = False
        break
if not equal_spacing:
    print("\nError: Intervals are not equally spaced. Trapezoidal rule
cannot be applied.")
else:
    total = y[0] + y[n]
    for i in range(1, n):
        if i % 3 == 0:
            total += 2 * y[i]
        else:
            total += 3 * y[i]
    result = (3 * h / 8) * total
    print("Value of integration =", ('%.6f' %
result).rstrip('0').rstrip('.'))
```

INPUT:

```
Enter number of intervals (multiple of 3): 6
Enter x and y values:
0 1
1 0.5
2 0.2
3 0.1
4 0.0588
5 0.0385
6 0.027
```

OUTPUT:

```
Value of integration = 1.357088
```


PYTHON CODE:

```
# Euler's Method
import math
equation = input("Enter dy/dx in terms of x and y: ")
def f(x, y):
    return eval(equation)
x = float(input("Enter initial value of x: "))
y = float(input("Enter initial value of y: "))
h = float(input("Enter step size h: "))
xn = float(input("Enter value of x at which y is required: "))
steps = int((xn - x) / h)
for i in range(steps):
    y = y + h * f(x, y)
    x = x + h
print("Approximate value of y =", ('%.6f' % y).rstrip('0').rstrip('.'))
```

INPUT:

```
Enter dy/dx in terms of x and y: x+y
Enter initial value of x: 0
Enter initial value of y: 1
Enter step size h: 0.1
Enter value of x at which y is required: 1
```

OUTPUT:

```
Approximate value of y = 3.187485
```

PYTHON CODE:

```
# Euler's Modified Method
import math
equation = input("Enter dy/dx in terms of x and y: ")
def f(x, y):
    return eval(equation)
x = float(input("Enter initial value of x: "))
y = float(input("Enter initial value of y: "))
h = float(input("Enter step size h: "))
xn = float(input("Enter value of x at which y is required: "))
steps = int((xn - x) / h)
for i in range(steps):
    yp = y + h * f(x, y) # predictor
    yc = y + (h / 2) * (f(x, y) + f(x + h, yp)) # corrector
    y = yc
    x = x + h
print("Approximate value of y =", ('%.6f' % y).rstrip('0').rstrip('.'))
```

INPUT:

```
Enter dy/dx in terms of x and y: x+y
Enter initial value of x: 0
Enter initial value of y: 1
Enter step size h: 0.1
Enter value of x at which y is required: 1
```

OUTPUT:

```
Approximate value of y = 3.428162
```

PYTHON CODE:

```
# Runge-Kutta 2nd Order Method
import math
equation = input("Enter dy/dx in terms of x and y: ")
def f(x, y):
    return eval(equation)
x = float(input("Enter initial value of x: "))
y = float(input("Enter initial value of y: "))
h = float(input("Enter step size h: "))
xn = float(input("Enter value of x at which y is required: "))
steps = int((xn - x) / h)
for i in range(steps):
    k1 = h * f(x, y)
    k2 = h * f(x + h, y + k1)
    y = y + (k1 + k2) / 2
    x = x + h
print("Approximate value of y =", ('%.6f' % y).rstrip('0').rstrip('.'))
```

INPUT:

```
Enter dy/dx in terms of x and y: x+y
Enter initial value of x: 0
Enter initial value of y: 1
Enter step size h: 0.1
Enter value of x at which y is required: 1
```

OUTPUT:

```
Approximate value of y = 3.428162
```

PYTHON CODE:

Runge-Kutta 4th Order Method

```
import math
equation = input("Enter dy/dx in terms of x and y: ")
def f(x, y):
    return eval(equation)
x0 = float(input("Enter initial value of x: "))
y0 = float(input("Enter initial value of y: "))
h = float(input("Enter step size h: "))
xn = float(input("Enter value of x at which y is required: "))
steps = int((xn - x0) / h)
x = x0
y = y0
for i in range(steps):
    k1 = h * f(x, y)
    k2 = h * f(x + h/2, y + k1/2)
    k3 = h * f(x + h/2, y + k2/2)
    k4 = h * f(x + h, y + k3)
    y = y + (k1 + 2*k2 + 2*k3 + k4) / 6
    x = x + h
print("Approximate value of y =", ('%.6f' % y).rstrip('0').rstrip('.'))
```

INPUT:

```
Enter dy/dx in terms of x and y: x+y
Enter initial value of x: 0
Enter initial value of y: 1
Enter step size h: 0.1
Enter value of x at which y is required: 1
```

OUTPUT:

```
Approximate value of y = 3.436559
```